

B.Tech Final-Year Project · Group C4

DocBlendAI

A Confidence-Aware Multi-Format Document Question-Answering Assistant

Project explanation: architecture, models, flowcharts and novelty

Guide: Prof. R. A. Tiwari

Department of Computer Science & Engineering

PRMIT&R, Badnera

Typed · Scanned / printed · Handwritten documents → trustworthy answers

Contents

1. Project in one page (problem, idea, result)
2. Novelty: what DocBlendAI adds
3. Use case diagram: who does what
4. Layered architecture: each layer explained
5. The seven synopsis modules
6. What happens internally when a document is uploaded
7. How text is extracted: typed, scanned, handwritten, tables
8. Confidence capture and calibration
9. How the LLM works (1): guarded refinement of extracted text
10. How the LLM works (2): retrieval-augmented question answering
11. Reliability tiers
12. Models used in the project
13. Data storage
14. Experience Center, exports and the user interface
15. Evaluation and results
16. Deployment, testing and engineering practice
17. Limitations and future work
18. Glossary

1. Project in one page

The problem

Students, researchers and faculty study from material in many forms: typed PDFs and slides, printed pages that were scanned or photographed with a phone, and handwritten class notes. Ordinary “chat with your PDF” tools assume clean digital text. When the text comes from OCR or handwriting recognition it contains reading errors, yet the tool answers just as confidently. The user cannot tell whether an answer rests on clean text or on a guess built from badly read words.

The idea

DocBlendAI reads any academic document, measures how well each line was read, and carries that measurement all the way to the answer. Every answer comes with a **reliability tier** (Certain Moderate Uncertain Unreadable) and with the exact page and lines it came from.

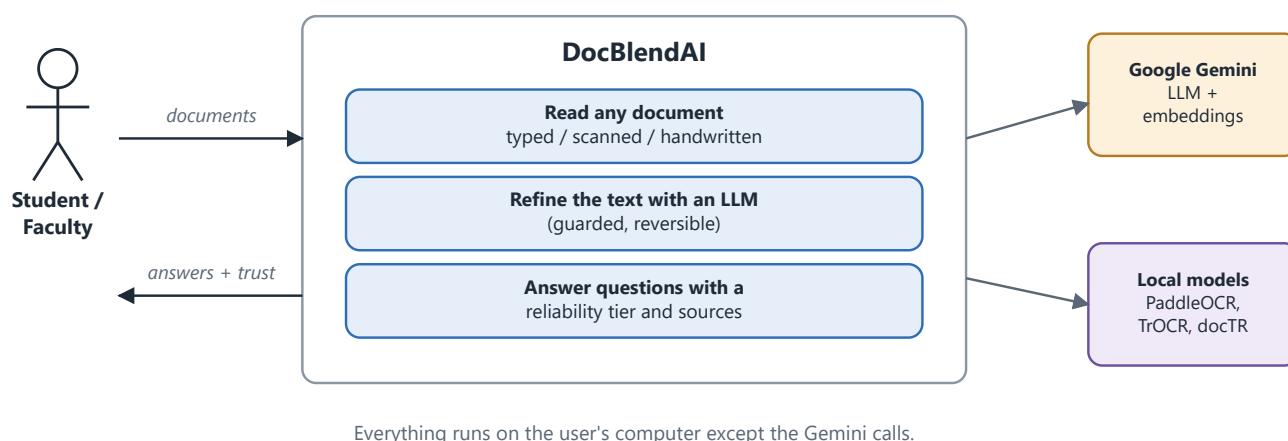


Figure 1. Context diagram: what goes in, what comes out, what runs where.

What the system does, step by step

1. **Detects the format** of an uploaded file (typed, scanned/printed, handwritten) and corrects the page orientation.
2. **Extracts the text** with the right method: exact parsing for digital files, PaddleOCR for printed pages, PaddleOCR line detection + TrOCR reading for handwriting, and PP-StructureV3 for layout and tables.
3. **Calibrates the confidence** of each recognised line so that, for example, 0.9 really means “about 90% right”.
4. **Refines the text with an LLM** (Gemini) automatically, but under strict guards, keeping the original so any change can be reverted.
5. **Indexes** the text in chunks with page/line citations, content type (table / paragraph / image) and confidence.
6. **Answers questions** with confidence-aware retrieval (similarity *and* readability) and Gemini, and labels each answer with a reliability tier.

In one sentence: DocBlendAI is a retrieval-augmented question-answering system that treats “*how well was this text read?*” as a first-class signal, from the OCR engine to the final answer.

Scope

In scope: typed, scanned/printed and handwritten academic documents uploaded as PDF, images (JPG/PNG/TIFF/BMP/WEBP, including phone photos), Word, PowerPoint, Excel or text files. **Out of scope:** live classroom transcription, non-Latin scripts, video.

Technology stack

Area	Technology
Backend	Python 3.11, FastAPI (one monolithic application), Uvicorn
Recognition	PaddleOCR 3.7 (PP-OCRv5, PP-StructureV3), TrOCR (Hugging Face transformers), docTR, Tesseract, OpenCV
Language model	Google Gemini via the google-genai SDK: gemini-3.5-flash-lite (answers, refinement), gemini-embedding-001 (vectors)
Storage	SQLite through SQLAlchemy (relational data), ChromaDB (vectors)
Frontend	Plain HTML, CSS and JavaScript (no framework), served by FastAPI
Evaluation	jiwer (CER, WER), own calibration metrics (ECE, MCE, Brier, NLL)
Deployment	Python virtual environment or Docker / Docker Compose

2. Novelty: what DocBlendAI adds

Existing tools usually do *one* of these things: OCR (Tesseract, PaddleOCR), handwriting recognition (TrOCR) or question answering over digital text (RAG chatbots). None of them carries the recognition uncertainty into the answer. The table compares DocBlendAI with a typical “chat with PDF” system.

Aspect	Typical RAG chatbot	DocBlendAI
Input formats	Digital PDF text only	Typed, scanned, photographed, handwritten; PDF, images, Office, text
Recognition errors	Ignored (scanned PDFs often give no text)	Measured per line, calibrated, shown to the user
Ranking of passages	Similarity only	Similarity + calibrated confidence
Trust in the answer	None shown	4-tier reliability + cited page/lines + readability
LLM use on the text	None, or silent rewriting	Guarded refinement: diffed, capped, reversible, never raises trust
Handwriting	Not supported	Paddle detection + TrOCR reading, ruled-line removal
Tables / layout	Lost (flat text)	Detected, kept as rows/cells, layout view and export
Unanswerable questions	Often hallucinated	NOT_FOUND protocol, labelled at least Uncertain

Novel contributions in detail

N1. Confidence-aware retrieval (core idea)

Normal RAG ranks passages by semantic similarity only. DocBlendAI ranks by $\text{combined} = 0.7 \times \text{similarity} + 0.3 \times \text{calibrated confidence}$, so a clearly read passage beats an equally relevant but badly recognised one. Recognition quality becomes part of retrieval, not an afterthought.

N2. Four-tier reliability label on every answer

The answer is labelled Certain / Moderate / Uncertain / Unreadable by weighing *both* retrieval relevance and recognition confidence of the evidence. The user sees why, through cited sources with similarity and readability values.

N3. Calibrated confidence that means the same for every engine

Raw scores of Tesseract, TrOCR and PaddleOCR are not comparable (TrOCR is over-confident). DocBlendAI fits temperature scaling (Ayllon et al., ICDAR 2024), Platt scaling and isotonic regression, chooses the one with the lowest validation Expected Calibration Error (ECE), and reports ECE, MCE, Brier score and NLL before and after. One threshold set then works for all formats.

N4. Hybrid handwriting recognition

PaddleOCR is used as a line *detector*, TrOCR (trained on handwriting) as the line *reader*. The engine is chosen automatically by comparing calibrated confidences. Ruled notebook lines are erased with OpenCV first. Example: Paddle alone read “Opera+ing systems”; the hybrid read “Operating systems”.

N5. Guarded, reversible automatic LLM refinement

Gemini corrects recognition errors line by line, but: it must answer in JSON per line; each change is word-diffed and labelled *corrected* or *inferred*; a line changed by more than 40% is rejected; the original is kept as a version (revert / reapply / retry); and LLM-written text can never raise reliability (capped at 0.84, i.e. at most Moderate). When Gemini is unavailable the system keeps working with the unrefined text.

N6. Format-aware unified extraction

One pipeline detects whether a file is typed, scanned or handwritten and routes it to exact parsing, OCR or HTR, while producing the same output structure (pages, lines, boxes, words, confidences, blocks, tables) for all of them.

N7. Content-type identification from layout

PP-StructureV3 layout blocks decide whether a chunk is a table, paragraph or image; tables keep their rows and cells, so table questions retrieve whole rows. A rule-based classifier is the fallback.

N8. Traceable answers: page and line citations

Each chunk stores page, first line and last line, so every source under an answer says exactly where it came from, and words corrected by the LLM are marked in the source.

N9. One shared document library, two views

The QA page and the Experience Center (OCR studio) work on the same documents and versions: a correction made in one is re-indexed and used by the other immediately.

N10. Layout-preserving view and export

Text is shown and exported in its original position on the page (columns, tables), not as a flat paragraph.

3. Use case diagram: who does what

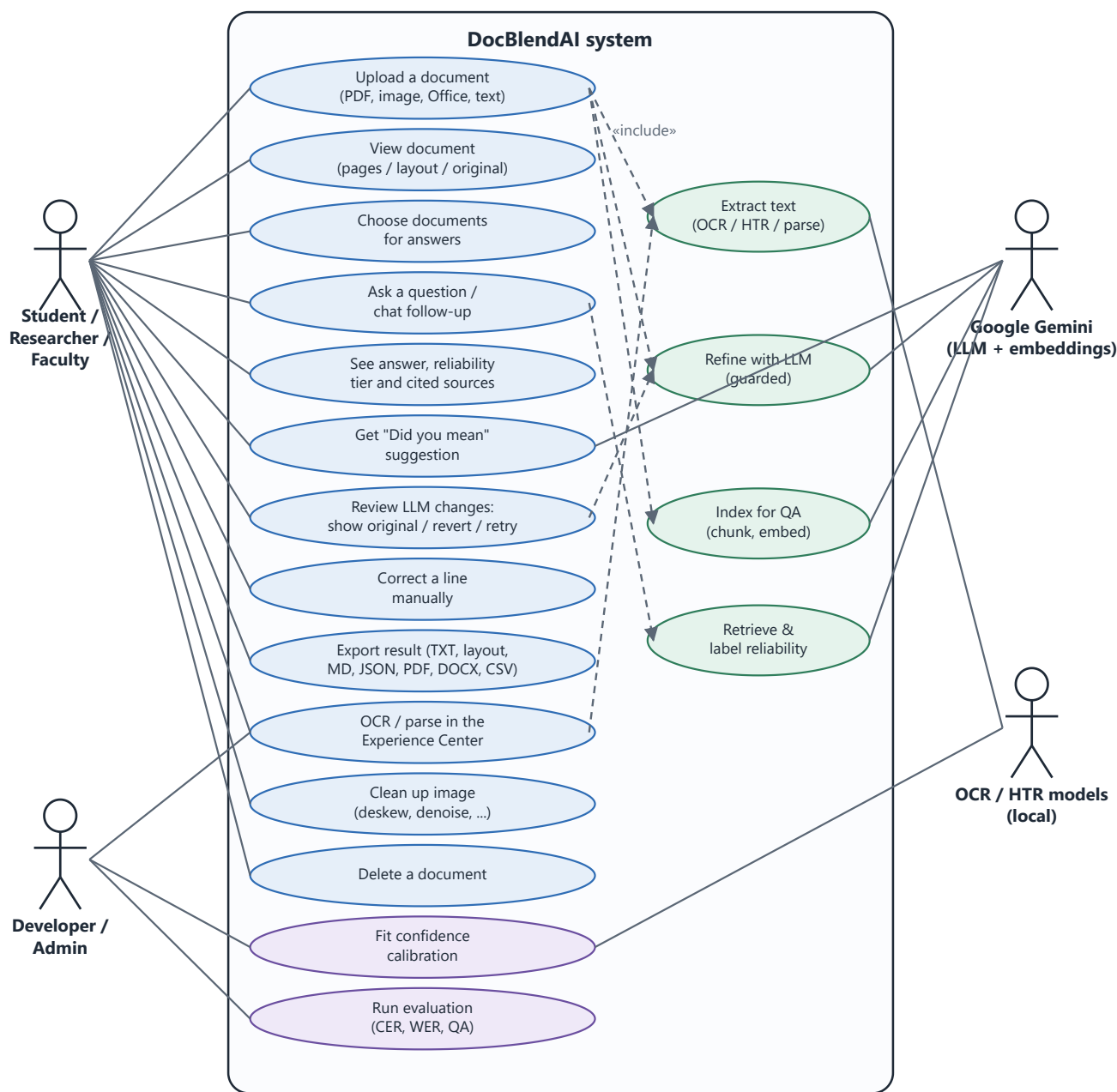


Figure 2. Use case diagram. Blue: user goals; purple: developer goals; green: internal use cases included by them.

Actors

Actor	Role
Student / Researcher / Faculty	Uploads documents, reads them, asks questions, checks sources, reviews LLM changes, exports text.
Developer / Admin	Uses the Experience Center for detailed OCR work, fits calibration on labelled data, runs the evaluation.
Google Gemini (external)	Creates embeddings, refines recognised text, writes answers, suggests spelling corrections.
OCR / HTR models (local)	PaddleOCR, PP-StructureV3, TrOCR, docTR, Tesseract running on the user's computer.

Main use cases

Use case	What happens internally
Upload a document	File is checked and saved; a background job extracts the text, refines it with Gemini, chunks, embeds and indexes it. Progress is shown live ("Extracting page 2 of 5...", "Refining with Gemini...", "Indexing for questions...", "Ready").
View document	Page images with recognised line boxes, a layout view that keeps the original positions, or the original file.
Choose documents	Tick-boxes limit retrieval to the selected documents (ChromaDB metadata filter on doc_id).
Ask / chat	Embedding, vector search, confidence-aware re-ranking, reliability tier, Gemini answer; the last 3 turns are sent for follow-ups.
See answer & sources	Answer text, tier badge, and up to 5 sources with document, page, lines, similarity and readability.
"Did you mean"	Misspelled question words are matched against the document vocabulary (fuzzy), optionally refined by Gemini.
Review LLM changes	Changed words are highlighted; the user can show the original, revert, use the refined text again or retry refinement.
Correct a line	Manual edit in the Experience Center; saved as a new version and the document is re-indexed.
Export	TXT, layout TXT, Markdown, JSON, searchable PDF, DOCX, CSV (tables).
Clean up image	Deskew, denoise, contrast, binarise, remove ruled lines before OCR.
Fit calibration	Labelled lines are used to fit temperature / Platt / isotonic calibrators; best by validation ECE.
Run evaluation	Scripts in evaluation/ compute CER, WER, Exact/Semantic match, calibration error and tier accuracy.

4. Layered architecture: each layer explained

DocBlendAI is a **single FastAPI monolith**. Modules are separated by Python files and folders, not by network services, which keeps setup and the demo simple while four team members could still work on separate modules.

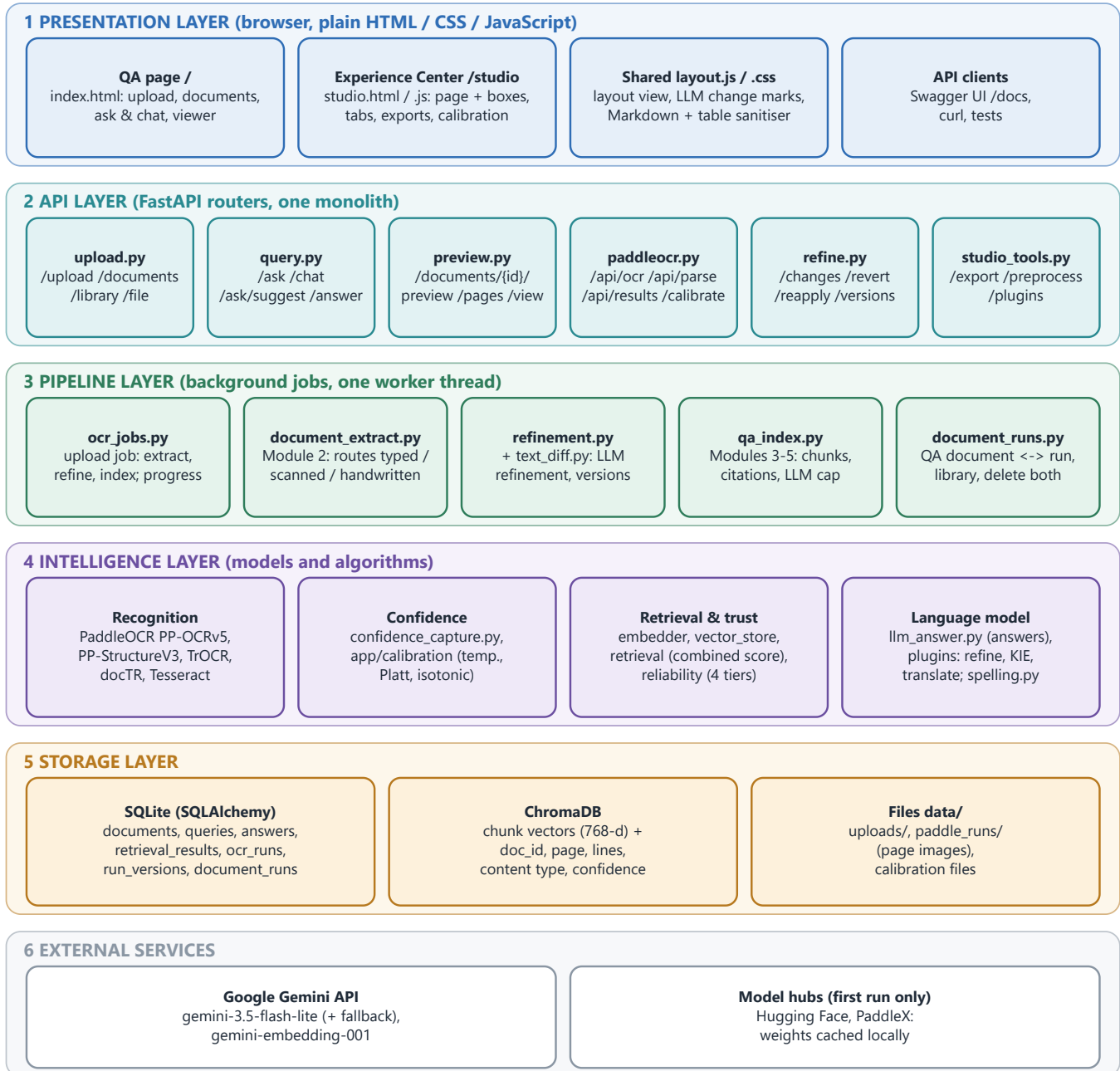


Figure 3. The six layers of DocBlendAI. Requests flow downwards; results flow back up.

Layer 1: Presentation

Two web pages written in plain HTML/CSS/JavaScript and served by FastAPI itself (no Node build step):

- **QA page** (/ , index.html): upload, document list with tick-boxes and live progress, document viewer (pages with boxes, layout view, original), chat with reliability badges and sources, “LLM changes” panel.
- **Experience Center** (/studio): detailed OCR workspace: page image with clickable line boxes, tabs for text, layout, Markdown, tables, JSON, confidence, versions; image clean-up; exports; calibration; plugins.

- **Shared layout module** (`layout.js`): draws text in its page position, marks LLM-corrected and inferred words, and safely renders Markdown and HTML tables (sanitised).

Layer 2: API

FastAPI routers expose REST endpoints with automatic validation (Pydantic) and documentation (`/docs`):

Router	Main endpoints	Purpose
<code>upload.py</code>	POST <code>/upload</code> , GET <code>/documents</code> , <code>/documents/{id}/file</code> , DELETE	Upload, list, download, delete documents
<code>query.py</code>	POST <code>/ask</code> , <code>/chat</code> , <code>/ask/suggest</code> ; GET <code>/answer/{id}/sources</code>	Question answering and sources
<code>preview.py</code>	<code>/documents/{id}/preview</code> , <code>/pages</code> , <code>/view</code>	Page images and viewer data
<code>paddleocr.py</code>	<code>/api/ocr</code> , <code>/api/parse</code> , <code>/api/results</code> , <code>/api/calibrate</code>	Experience Center OCR / parsing jobs, calibration
<code>refine.py</code>	<code>/changes</code> , <code>/revert</code> , <code>/reapply</code> , <code>/refine</code> , <code>/versions</code>	LLM changes and version history
<code>studio_tools.py</code>	<code>/export</code> , <code>/preprocess</code> , <code>/plugins</code>	Exports, image clean-up, plugins (refine, KIE, translate)

Layer 3: Pipeline

- **ocr_jobs.py**: a job manager with one worker thread. Each upload becomes a job that runs extraction → refinement → indexing and reports progress. Heavy models are loaded once and reused.
- **document_extract.py**: Module 2, chooses typed / scanned / handwritten extraction and returns one common result structure.
- **refinement.py** + **text_diff.py**: the guarded LLM refinement and the word-level diff.
- **qa_index.py**: turns a result into chunks with content type, confidence (LLM cap) and page/line citation, then embeds and stores them.
- **document_runs.py**: links a QA document to its Experience Center run, so both pages share one library.

Layer 4: Intelligence (models and algorithms)

- **Recognition**: PaddleOCR (PP-OCRv5 text detection + recognition), PP-StructureV3 (layout, tables, formulas), TrOCR (handwriting), docTR and Tesseract (classic engine), OpenCV preprocessing.
- **Confidence**: `confidence_capture.py` (raw scores per engine, temperature scaling) and `app/calibration` (temperature, Platt, isotonic; ECE-based selection).
- **Retrieval and trust**: `embedder.py` (Gemini embeddings), `vector_store.py` (ChromaDB), `retrieval.py` (combined score), `reliability.py` (4 tiers).
- **Language model**: `llm_answer.py` (grounded answers, fallback model), plugins (refine, key-information extraction, translate), `spelling.py` ("Did you mean").

Layer 5: Storage

SQLite keeps everything relational (documents, questions, answers, retrieval results, OCR runs, versions); **ChromaDB** keeps only the chunk vectors and their metadata; the **data/** folder keeps uploaded files, rendered page images and calibration files. SQLite needs no server, so every team laptop runs the same setup.

Layer 6: External services

Only Google Gemini is called over the network (embeddings, refinement, answers). Model weights for PaddleOCR, TrOCR and docTR are downloaded once from their hubs and cached; after that recognition runs offline.

5. The seven synopsis modules

The approved synopsis defines seven modules. The code keeps the same names and boundaries.

#	Module	Main files	Responsibility
1	Document Upload	routers/upload.py, file_types.py	Accept PDF, images, Office, text; check type/size; store; start the job
2	Format Detection & Text Extraction	document_extract.py, text_parser.py, paddleocr_service.py, htr_extractor.py, ocr_extractor.py, preprocess.py, line_segmentation.py	Decide typed / scanned / handwritten; route to parsing / OCR / HTR; correct orientation
3	Confidence Capture & Calibration	confidence_capture.py, app/calibration/	Raw line confidence per engine; calibrated probability
4	Content-Type Identification	content_type.py, qa_index.py	Table / paragraph / image for each chunk (layout first, rules as fallback)
5	Confidence-Aware Retrieval (RAG)	embedder.py, vector_store.py, retrieval.py	Embed, search, re-rank by combined score
6	Reliability Tier Classification	reliability.py	Certain / Moderate / Uncertain / Unreadable
7	LLM Answer Generation	llm_answer.py	Grounded Gemini answer, NOT_FOUND, fallback model

Synopsis entities (unchanged in `schemas.py`)

Entity	Fields
Document	doc_id, file_path, format_type, page_count
RecognizedChunk	chunk_id, text, content_type, raw_conf, calibrated_conf, vector
Query	query_id, question_text, user_id
RetrievalResult	chunk_id, similarity, confidence, combined_score
Answer	answer_id, answer_text, reliability_label

6. What happens internally when a document is uploaded

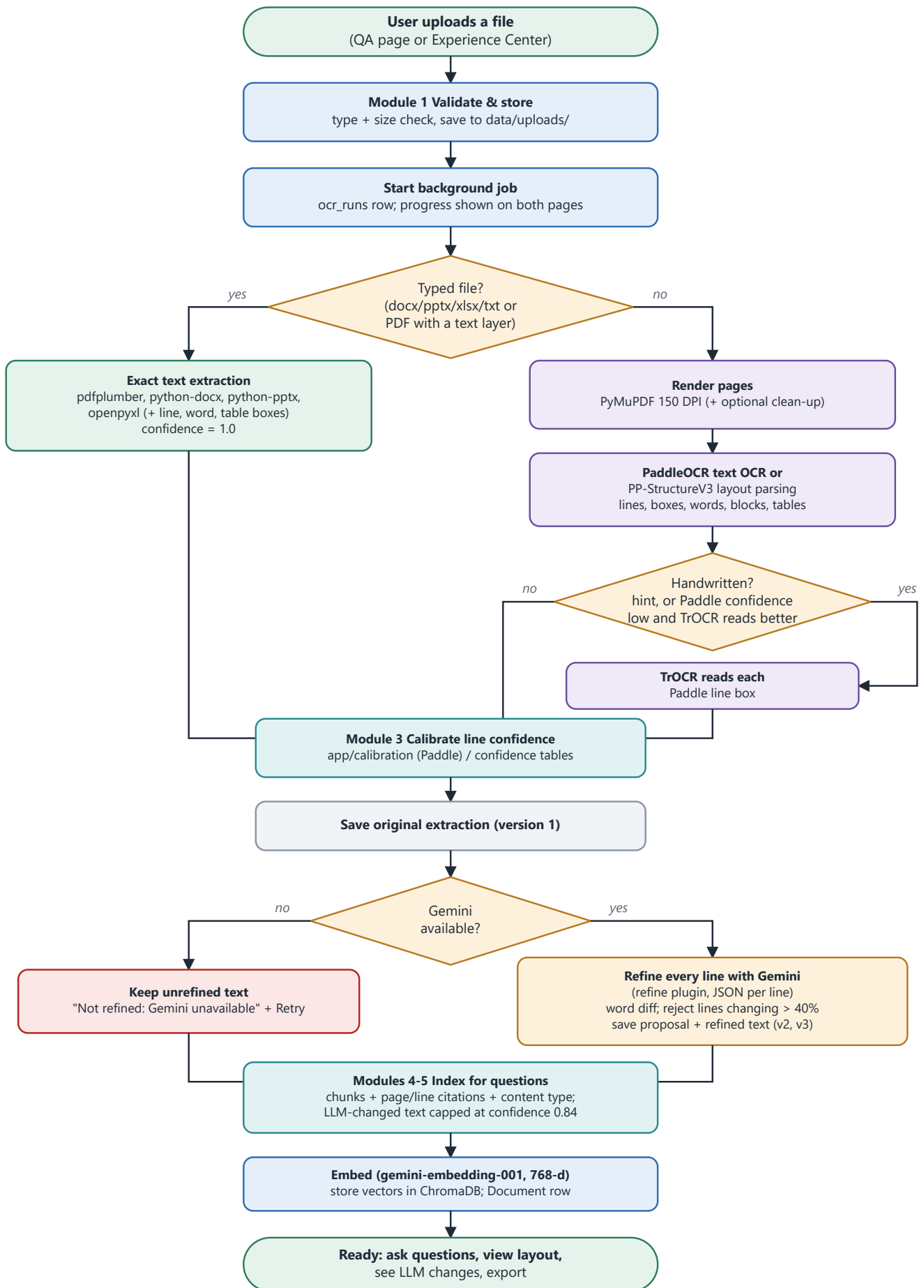


Figure 4. Upload pipeline: one background job per document.

Explanation of each step

- 1. **Validate and store.** The extension and size are checked against `file_types.py`; the file is saved under `data/uploads/` with a new `doc_id`.
- 2. **Background job.** The request returns immediately; a job (an `ocr_runs` row) runs in a worker thread. Both pages poll its progress, so the user sees the document appear with “Extracting page i of n...”.
- 3. **Typed check.** Word, PowerPoint, Excel, text files and PDFs that already contain a text layer are *typed*: their text is read exactly (confidence 1.0), with line and word boxes and tables from pdfplumber.
- 4. **OCR path.** Other PDFs are rendered at 150 DPI with PyMuPDF; images are loaded directly. Orientation is corrected and PaddleOCR (or PP-StructureV3 for layout and tables) recognises lines with boxes and scores.
- 5. **Handwriting decision.** If the user marked the file as handwritten, or Paddle's calibrated confidence is low (< 0.6), TrOCR reads a sample of Paddle's line boxes. Whichever engine reaches the higher calibrated confidence wins.
- 6. **Calibration.** Every line gets a raw and a calibrated confidence (Section 8).
- 7. **Original version.** The unrefined result is saved as version 1, so it can always be restored.
- 8. **Guarded LLM refinement** (Section 9). If Gemini is unavailable the document is still usable, with a clear “Not refined” notice and a Retry link.
- 9. **Indexing.** Text is split into chunks of about 1000 characters with 200 overlap, each with page/line citation, content type and confidence; LLM-written text counts at most 0.84.
- 10. **Embedding and storage.** Chunks are embedded (768-dimension vectors) and stored in ChromaDB; the `Document` row is written to SQLite. The document is now “Ready”.

Format detection rules

Evidence	Detected format	Extraction method
Office / text file, or PDF with a usable text layer	typed	Direct parsing (exact text)
Image or image-only PDF; Paddle mean calibrated confidence ≥ 0.6	scanned (printed)	PaddleOCR / PP-StructureV3
Paddle confidence low and TrOCR reads the boxes better, or user hint	handwritten	Paddle line detection + TrOCR

7. How text is extracted

7.1 Typed documents (exact parsing)

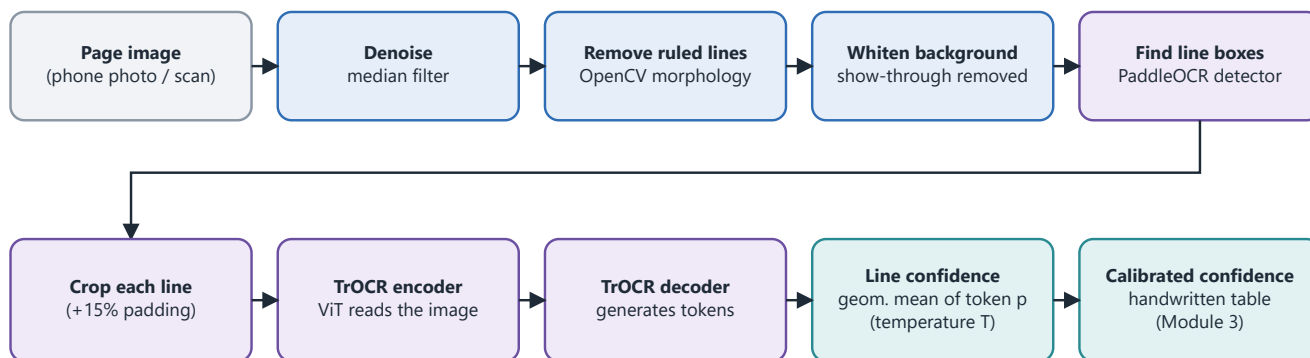
- **PDF with text layer:** pdfplumber gives every word with its box; words are grouped into lines; tables are detected from ruling lines.
- **Word (.docx):** python-docx reads paragraphs and tables in order. **PowerPoint (.pptx):** python-pptx reads each slide's text frames as a page. **Excel:** openpyxl reads each sheet as a table. **Text:** read as is.
- Confidence is 1.0 because no recognition is involved. The exact text is used for QA unless the user edits a line.

7.2 Scanned and photographed printed pages (OCR)

1. **Render / load:** PyMuPDF renders each PDF page at 150 DPI; images are read with OpenCV.
2. **Preprocess (optional):** deskew, denoise, contrast (CLAHE), binarise; page orientation is fixed by PaddleOCR's document-orientation classifier (PP-LCNet).
3. **Text detection:** PP-OCrv5 detector (DB, a differentiable-binarisation segmentation network) predicts a text-probability map and turns it into line polygons.
4. **Text-line orientation:** PP-LCNet classifier flips upside-down lines.
5. **Recognition:** PP-OCrv5 recogniser (SVTR-style network with CTC decoding) reads each cropped line and returns text and a score.
6. **Words:** character positions from the recogniser are used to give each word its own box.

7.3 Layout, tables and formulas (PP-StructureV3)

- **Layout detection** (PP-DocLayout_plus-L) finds blocks: title, paragraph, table, figure, formula, header, footer ...; PP-DocBlockLayout orders them for multi-column reading order.
- **Tables:** a classifier decides wired/wireless; RT-DETR cell detectors and SLANeXt / SLANet_plus structure models rebuild the table as HTML rows and cells.
- **Formulas:** PP-FormulaNet_plus-S converts formulas to LaTeX.
- The output gives Markdown for the page, blocks with boxes, and tables; DocBlendAI uses the blocks to set the **content type** of each chunk.



Why this design: PaddleOCR is a strong line DETECTOR on any page, while TrOCR (trained on handwriting) is the better line READER for cursive text. DocBlendAI combines them: Paddle finds where each line is, TrOCR reads it. On a test note Paddle alone read "Opera+ing systems"; Paddle boxes + TrOCR read "Operating systems" correctly.

Figure 5. Handwriting pipeline: Paddle finds the lines, TrOCR reads them.

7.4 Handwritten notes (HTR)

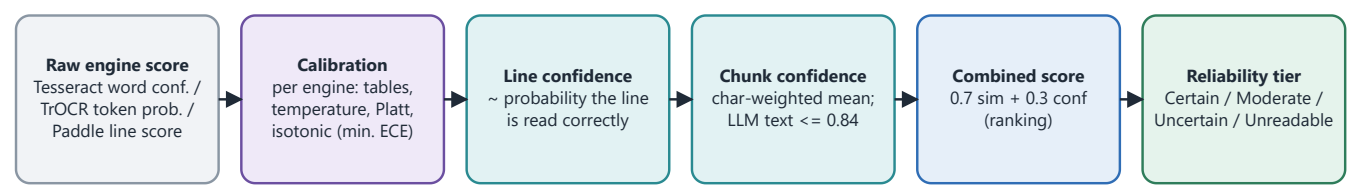
TrOCR (`microsoft/trocr-small-handwritten`) is a transformer encoder–decoder: a Vision Transformer encoder splits the line image into 16×16 patches and encodes them; a text decoder generates the line one token at a time, attending to the image. For each token the decoder gives a probability; the line confidence is their geometric mean after **temperature scaling** (dividing the logits by $T > 1$), which removes TrOCR's over-confidence.

Before TrOCR, OpenCV **removes ruled notebook lines** (long horizontal / vertical structures found by morphological opening) and whitens the background, because ruled lines were the main cause of gibberish on real notebook pages.

7.5 Classic engine (alternative)

The original pipeline (`ingest_pipeline=classic`) uses **Tesseract** for printed text (or **docTR** `db_resnet50 + crnn_vgg16_bn` when Tesseract is not installed), and docTR detection + TrOCR for handwriting. It is still available and tested, and is used as a baseline in the evaluation.

8. Confidence capture and calibration



Every number the user sees as "readability" has the same meaning across engines: after calibration a 0.9 means "about 9 in 10 characters / lines are right", whether the page was typed, printed, scanned or handwritten. That is what lets one ranking formula and one set of reliability thresholds work for all formats.

Figure 6. From a raw engine score to a reliability tier.

Why calibration is needed

Each engine reports a score, but the scores mean different things. TrOCR often reports 0.99 for a wrong line; Tesseract word confidences are on a 0–100 scale; Paddle scores are high even for slightly wrong lines. A model is **well calibrated** when, among all lines with confidence 0.8, about 80% are correct. Only then can one formula and one set of thresholds serve every format.

Methods implemented (app/calibration)

Method	Formula / idea	Notes
Temperature scaling	$p = \sigma(\text{logit}(s) / T)$, one parameter T	Ayllon et al., ICDAR 2024 (synopsis ref. [3]); also applied inside TrOCR on token logits
Platt scaling	$p = \sigma(a \cdot \text{logit}(s) + b)$	Two parameters fitted by logistic regression
Isotonic regression	monotone step function (pool-adjacent-violators)	Add-one prior keeps it away from exact 0 and 1
Per-format tables	piece-wise linear knots, e.g. scanned 0.70→0.73, 0.94→0.99	Used by the classic engine (data/calibration.json)

The data are split into fitting and validation parts; the method with the lowest **validation ECE** is kept.

Metrics

- **ECE** (Expected Calibration Error) = $\sum_{\text{bins}} (n_b/N) \cdot |\text{accuracy}_b - \text{confidence}_b|$
- **MCE** (Maximum Calibration Error) = the largest of those bin gaps.
- **Brier score** = mean of $(p - y)^2$; **NLL** = mean of $-[y \log p + (1-y) \log(1-p)]$.

Example result (sample PaddleOCR run, 119 labelled lines, 76% correct)

Metric	Before	After (isotonic, chosen)
ECE	0.131	0.121
MCE	0.895	0.375
Brier	0.103	0.050

Metric	Before	After (isotonic, chosen)
NLL	0.296	0.199

The classic engine's per-format calibration lowered calibration error from 0.062 to 0.013 (scanned) and from 0.061 to 0.050 (handwritten) on the evaluation set.

9. How the LLM works (1): guarded refinement of extracted text

OCR and HTR make small reading errors ("process", "Opera+ing", "rn" for "m"). A large language model knows the language and can fix them, but it may also *invent* text. DocBlendAI therefore refines automatically but surrounds the LLM with guards.

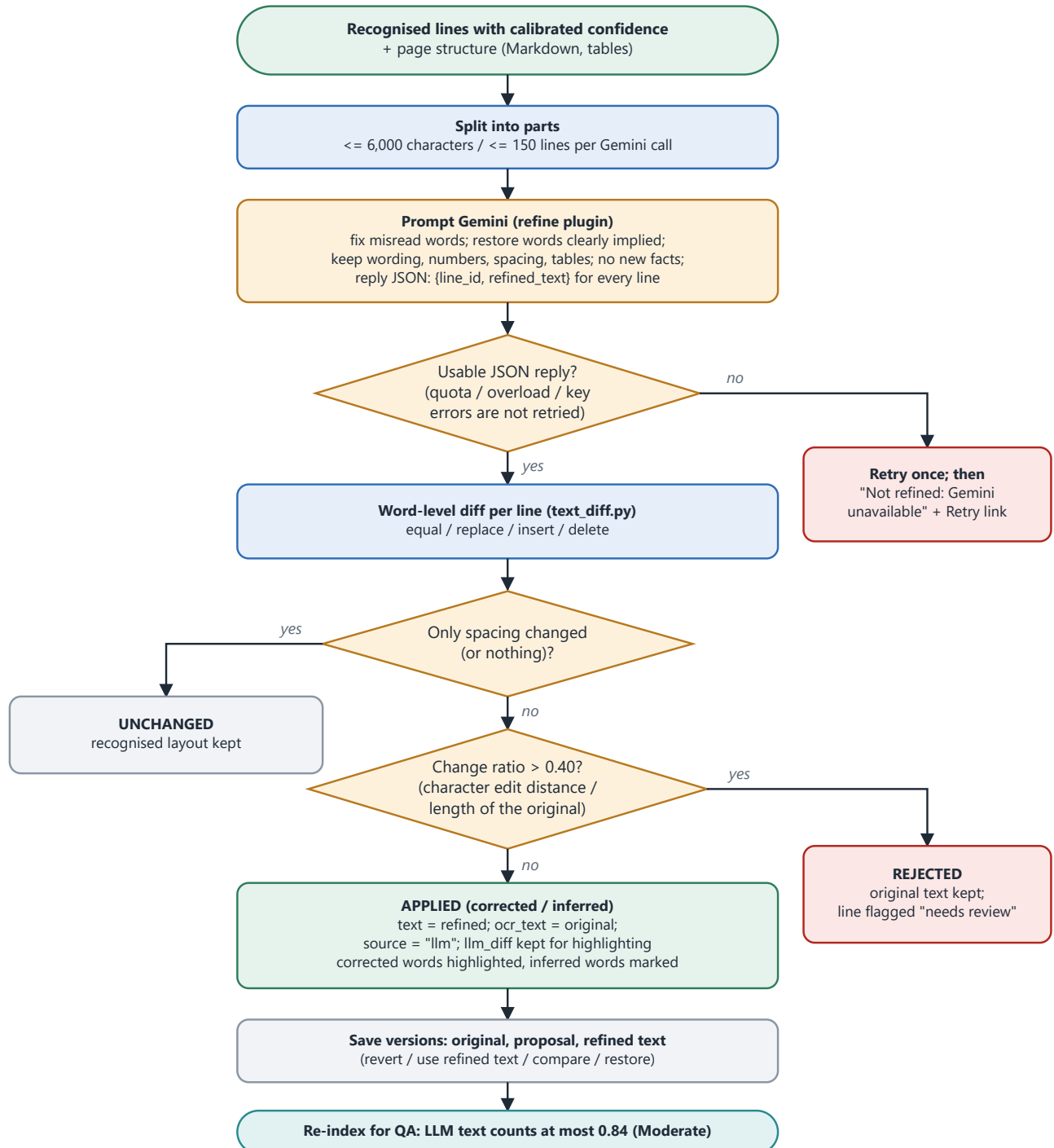


Figure 7. Guarded LLM refinement.

The guards

1. **Structured output:** Gemini receives numbered lines and must reply with JSON `{line_id, refined_text}` for every line; anything else is rejected. Lines cannot be merged, dropped or invented.

2. **Clear instructions:** fix misread words, restore words that are clearly implied, keep wording, numbers, spacing and tables, add no facts.
3. **Word diff:** each changed line is compared word by word; words are labelled *corrected* (replaced) or *inferred* (inserted), and highlighted in the UI.
4. **Change cap:** if more than 40% of a line changes, the change is rejected and the line is flagged “needs review”.
5. **Versions:** original extraction, LLM proposal and refined text are saved; the user can show the original, revert, reapply or retry.
6. **Trust cap:** LLM-written lines count at most **0.84** confidence in QA, so an answer that rests on them is at most **Moderate**; the LLM can never make an answer look *more* certain.
7. **Graceful failure:** quota (429), overload (503) or a missing key never block a document; it stays usable unrefined.

Live example: a .docx containing “proccess” was corrected to “process”. A question about it was answered **Moderate** (evidence is LLM text); after *Revert* to the exact original text, the same question was **Certain**.

10. How the LLM works (2): retrieval-augmented question answering

Retrieval-Augmented Generation (RAG) means the LLM does not answer from memory: the system first *retrieves* the most relevant passages from the user's documents and then asks the LLM to answer *only* from them. This keeps answers grounded and lets the system show the sources.

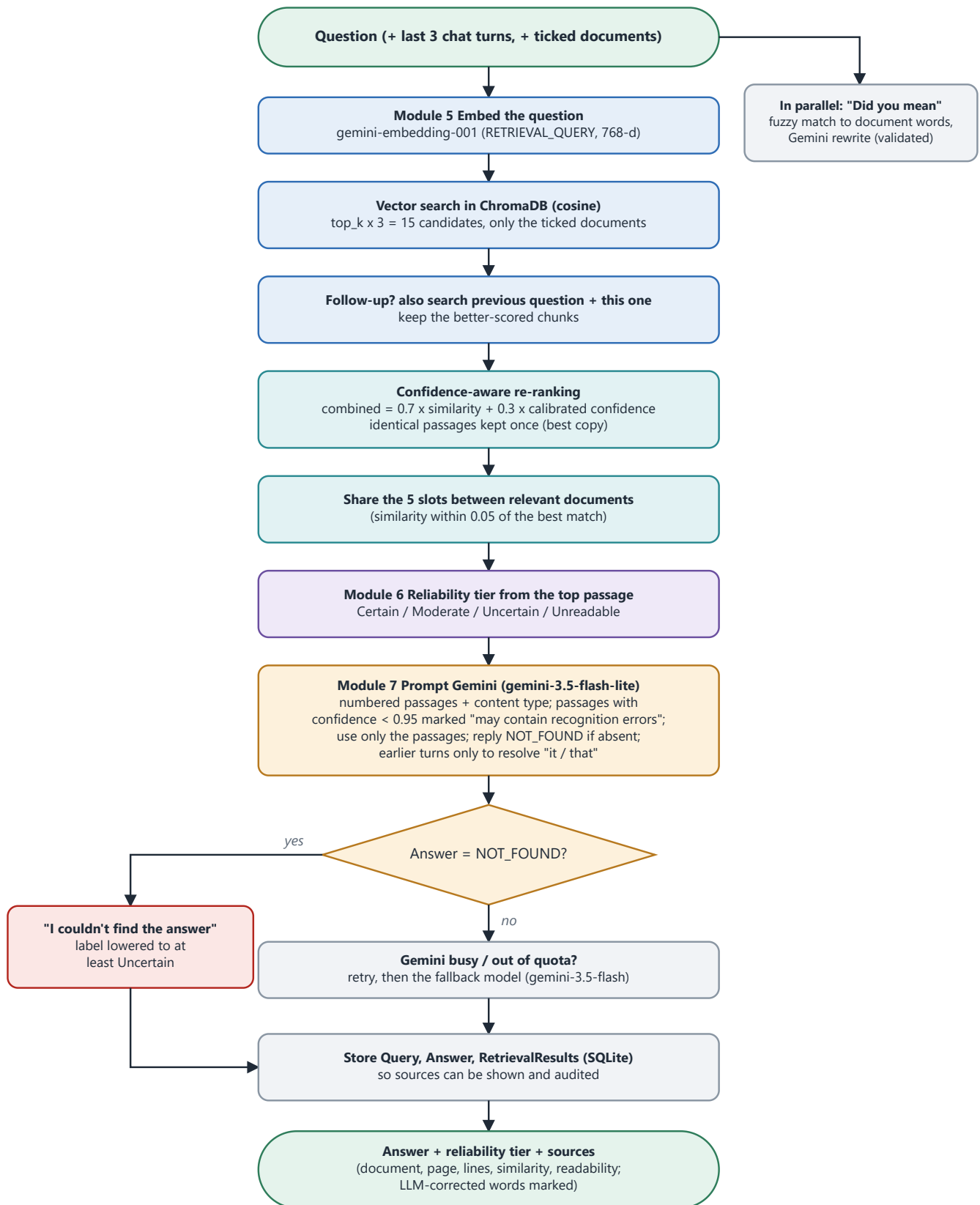


Figure 8. Question answering flow (Modules 5, 6 and 7).

Step by step

1. **Embedding.** gemini-embedding-001 turns the question into a 768-number vector. Chunks were embedded the same way when indexed, so texts with similar meaning have vectors pointing in similar directions.

2. **Vector search.** ChromaDB returns the 15 nearest chunks (cosine similarity), restricted to the ticked documents.
3. **Follow-ups.** In a chat, the previous question is searched together with the new one, so “explain it in more detail” still finds the right chunks.
4. **Confidence-aware re-ranking.** $\text{combined} = 0.7 \times \text{similarity} + 0.3 \times \text{calibrated confidence}$; duplicates removed; the 5 slots are shared fairly between documents that are almost equally relevant (within 0.05 similarity).
5. **Reliability tier** from the top passage (Section 11).
6. **Prompt.** Gemini (gemini-3.5-flash-lite) receives the numbered passages with their content type, a warning on passages read with confidence below 0.95, the last three chat turns (only to resolve “it” / “that”), and the rule: answer only from the passages, otherwise reply `NOT_FOUND`.
7. **Answer.** A `NOT_FOUND` reply becomes “I couldn't find the answer in the document” and the label is lowered to at least Uncertain. If the main model is out of quota or busy, the call is retried on the fallback model (gemini-3.5-flash).
8. **Store and show.** Query, Answer and RetrievalResults are stored in SQLite; the page shows the answer, tier and sources with page/line.

How the language model itself generates an answer

Gemini is a transformer language model. The prompt (instructions + passages + question) is split into tokens; self-attention layers let every token look at every other token, so the model links the question words to the matching passage words. It then generates the answer one token at a time, each time choosing a likely next token given everything before it. Because the passages are in the prompt, the most likely continuation is a summary of them; the instructions and the `NOT_FOUND` rule discourage invented facts.

Simplified answer prompt

```
You answer questions about the user's documents using ONLY the passages below.
Passages marked "may contain recognition errors" were read by OCR/HTR with lower confidence.
If the passages do not contain the answer, reply exactly NOT_FOUND.

[1] (paragraph, page 2) ... passage text ...
[2] (table, page 3, may contain recognition errors) ... passage text ...

Earlier conversation (only to understand references): ...
Question: What are the four reliability tiers?
```

11. Reliability tiers

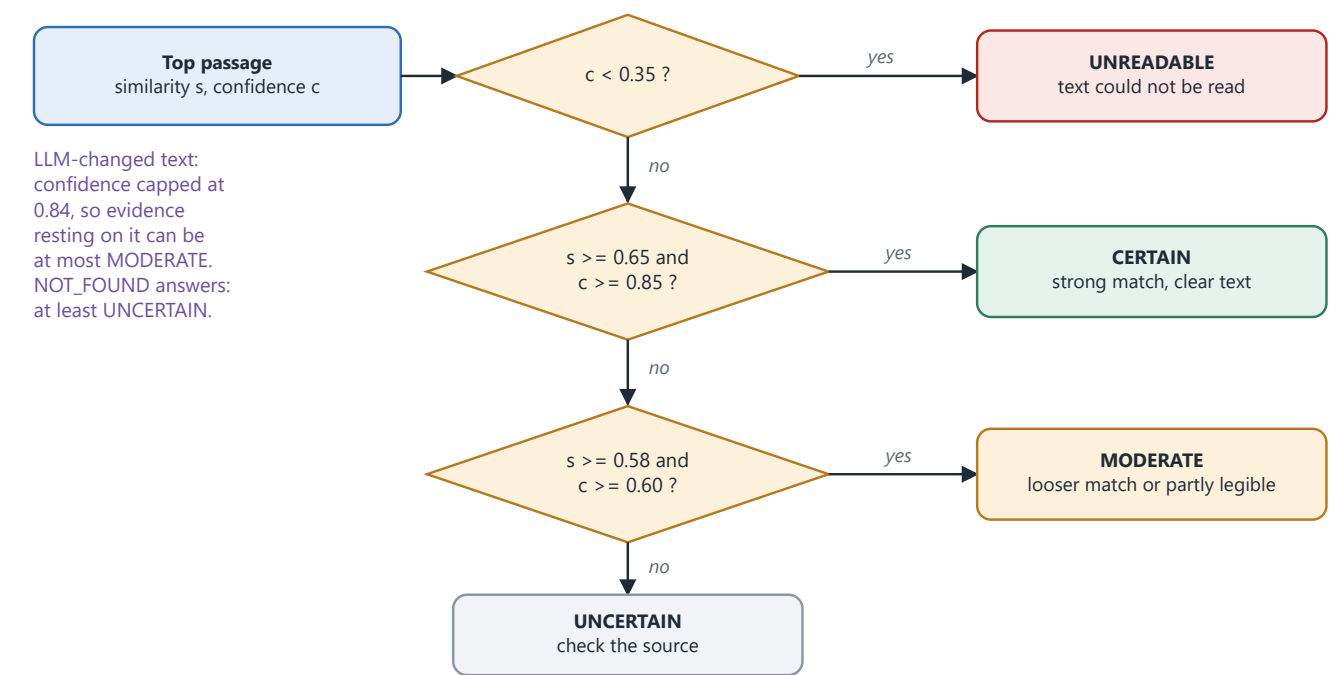


Figure 9. Reliability decision from the top retrieved passage.

Tier	Rule	Meaning for the user
Certain	similarity ≥ 0.65 and confidence ≥ 0.85	Strong match on clearly read text: trust it.
Moderate	similarity ≥ 0.58 and confidence ≥ 0.60	Looser match or partly legible / LLM-corrected text: probably right, glance at the source.
Uncertain	otherwise (or answer not found)	Weak evidence: check the source before using it.
Unreadable	confidence < 0.35	The relevant text could not be read reliably.

The thresholds were tuned on the evaluation set; the similarity thresholds reflect the range of gemini-embedding-001 cosine scores. Because confidence is calibrated, the same thresholds apply to typed, scanned and handwritten text.

12. Models used in the project

Model	Type / architecture	Used for	Where
PP-OCRv5 mobile detector	DB (differentiable binarisation) segmentation CNN	Finding text lines on printed and handwritten pages	paddleocr_service.py
PP-OCRv5 mobile recogniser	SVTR-based recogniser with CTC decoding	Reading printed lines	paddleocr_service.py
PP-LCNet classifiers	Lightweight CNN	Page orientation, text-line orientation, table type	PaddleOCR pipelines
PP-DocLayout_plus-L	RT-DETR object detector	Layout blocks (title, text, table, figure, formula ...)	PP-StructureV3
PP-DocBlockLayout	Detector	Reading order of multi-column pages	PP-StructureV3
SLANeXt_wired, SLANet_plus	Table structure recognition	Rows / columns / cells as HTML	PP-StructureV3
RT-DETR-L wired / wireless cell detectors	Transformer detectors	Table cell boxes	PP-StructureV3
PP-FormulaNet_plus-S	Encoder-decoder	Formulas to LaTeX	PP-StructureV3
PaddleOCR-VL (optional, off)	Vision-language model	Alternative document parsing	paddleocr_service.py
TrOCR small handwritten	ViT encoder + transformer decoder (~62M parameters)	Reading handwritten lines	htr_extractor.py
docTR db_resnet50 + crnn_vgg16_bn	DBNet detector + CRNN recogniser	Classic engine: line detection, OCR fallback	line_segmentation.py, ocr_extractor.py
Tesseract 5	LSTM OCR engine	Classic engine printed OCR	ocr_extractor.py
gemini-embedding-001	Embedding model (768-d output)	Chunk and question vectors	embedder.py
gemini-3.5-flash-lite	Large language model	Answers, refinement, spelling suggestions, plugins	llm_answer.py, plugins/
gemini-3.5-flash (fallback)	Large language model	Used when the main model is out of quota or busy	llm_answer.py

Non-learning algorithms

- OpenCV: median denoise, CLAHE contrast, Otsu / adaptive binarisation, deskew (minimum-area rectangle), ruled-line removal (morphology).
- Calibration: temperature, Platt, isotonic (PAV) with ECE-based selection.
- Word-level diff (sequence matching) and edit-distance change ratio for refinement.
- Fuzzy matching against the document vocabulary for “Did you mean”.
- Chunking with overlap and line-based citations; rule-based content-type classifier.

13. Data storage

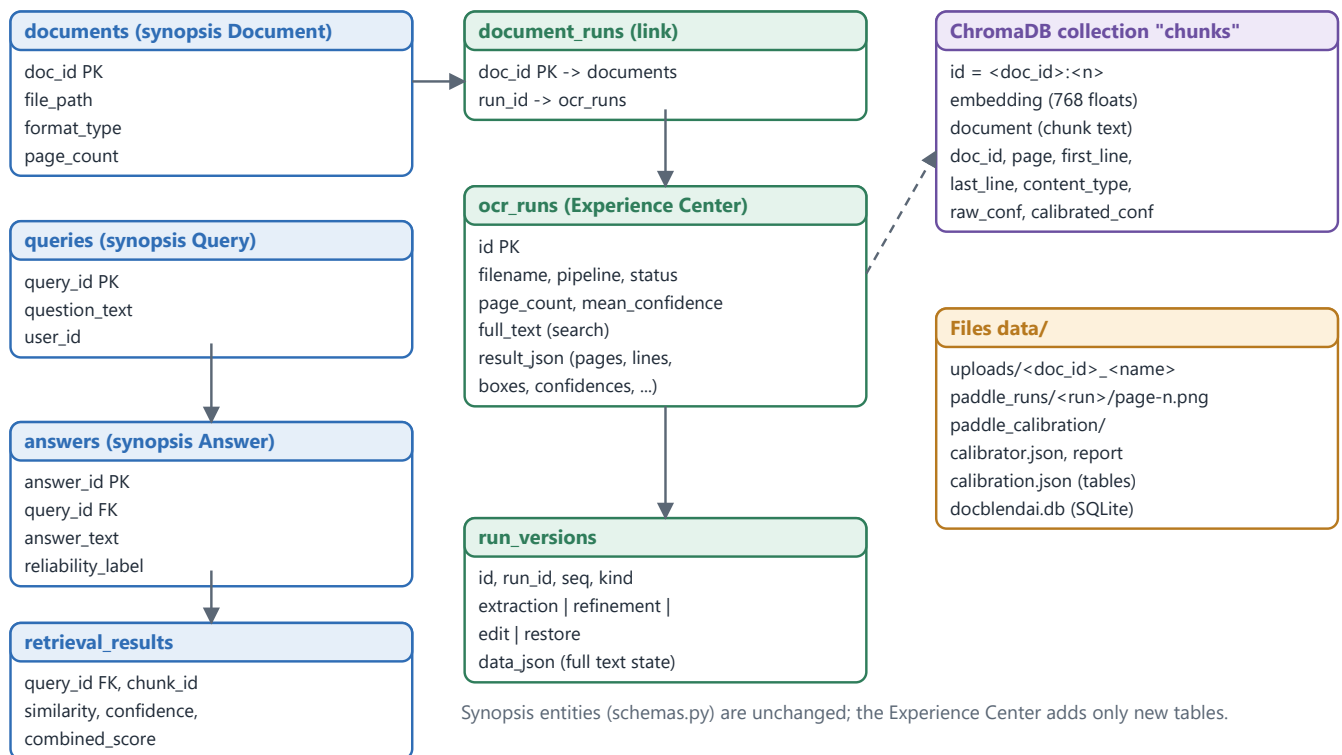


Figure 10. SQLite tables, the ChromaDB collection and the data folder.

- **Why two stores:** ChromaDB is good at nearest-neighbour search on vectors; SQLite is good at relations and filters (format type, reliability label, versions). Each does what it is best at.
- **Chunk id** <doc_id>:<n> links a retrieval result back to its document.
- **Versions:** every extraction, refinement, manual edit or restore adds a row to `run_versions`, so any earlier text can be brought back.
- **Deleting** a document removes its file, rows, run and vectors together.

14. Experience Center, exports and the user interface

QA page

- Drag-and-drop upload with optional “handwritten” hint; the document appears at once with live progress.
- Document list with tick-boxes to choose which documents answer questions; delete button.
- Viewer: page images with recognised boxes, layout view, or the original file.
- Chat: answers with tier badges, expandable sources (document, page, lines, similarity, readability), “Did you mean” suggestions, LLM-corrected words marked.
- “LLM changes” panel: each changed line with original and refined text; revert / reapply / retry.

Experience Center (/studio)

- Choose pipeline (text OCR or PP-StructureV3), engine options, image clean-up preview.
- Page canvas with coloured boxes by confidence; click a line to see raw and calibrated confidence and edit it.
- Tabs: Text, Layout, Markdown, Tables, JSON, Confidence (histogram and reliability diagram), Versions.
- Low-confidence lines (below 0.70) are flagged for review.
- Exports: TXT, layout-preserving TXT, Markdown, JSON, searchable PDF, DOCX, CSV.
- Calibration: upload labelled data, fit, see ECE/MCE/Brier/NLL before and after.
- Plugins: refine, key-information extraction, translate.

15. Evaluation and results

The synopsis metrics are **Exact Match**, **Semantic Match**, **Character Error Rate (CER)** and **Word Error Rate (WER)**, measured across the three formats. Scripts are in `evaluation/`; the report is `evaluation/results/report.md`. The current numbers come from a synthetic evaluation set (generated typed, scanned and handwritten pages with known text).

Metric definitions

- CER** = (substitutions + deletions + insertions) / characters in the reference; **WER** = the same on words.
- Exact Match**: normalised answer equals the reference. **Semantic Match**: embedding similarity with the reference above a threshold.

Recognition

Format	CER	WER	Format detected correctly
Typed	0.000	0.000	yes
Scanned / printed	0.366	0.504	71%
Handwritten	0.020	0.077	38%

Question answering

Question set	Exact	Contains	Semantic	Correct	Retrieval hit
Answerable	19%	81%	88%	88%	88%
Unanswerable	–	–	–	100% (NOT_FOUND)	–

Reliability tiers vs correctness

Tier	Answers	Correct
Certain	14	100%
Uncertain	4	50%

The tiers behave as intended: **Certain** answers were always right, while **Uncertain** answers were right only half the time, so the label tells the user when to check.

Noise robustness (OHRBench idea, synopsis ref. [2])

`evaluation/noise_robustness.py` injects graded OCR noise into the text and checks that reliability labels drop as noise rises.

Live checks on real files

File	Result	Time (CPU)
Typed PDF	Exact text, Certain answers	3.5 s
Word file with typo	Typo corrected by Gemini; Moderate, Certain after revert	3 s
Scanned page	PaddleOCR text + boxes	92 s

File	Result	Time (CPU)
Table image	Table parsed into correct rows/cells by PP-StructureV3	63 s
Handwritten note	Read correctly by Paddle boxes + TrOCR	40 s

Note: scanned CER on the synthetic set is high because those pages were generated with heavy noise and blur; real scans read much better. Format detection for handwriting is conservative: unclear cases fall back to OCR, and the user can set the “handwritten” hint.

16. Deployment, testing and engineering practice

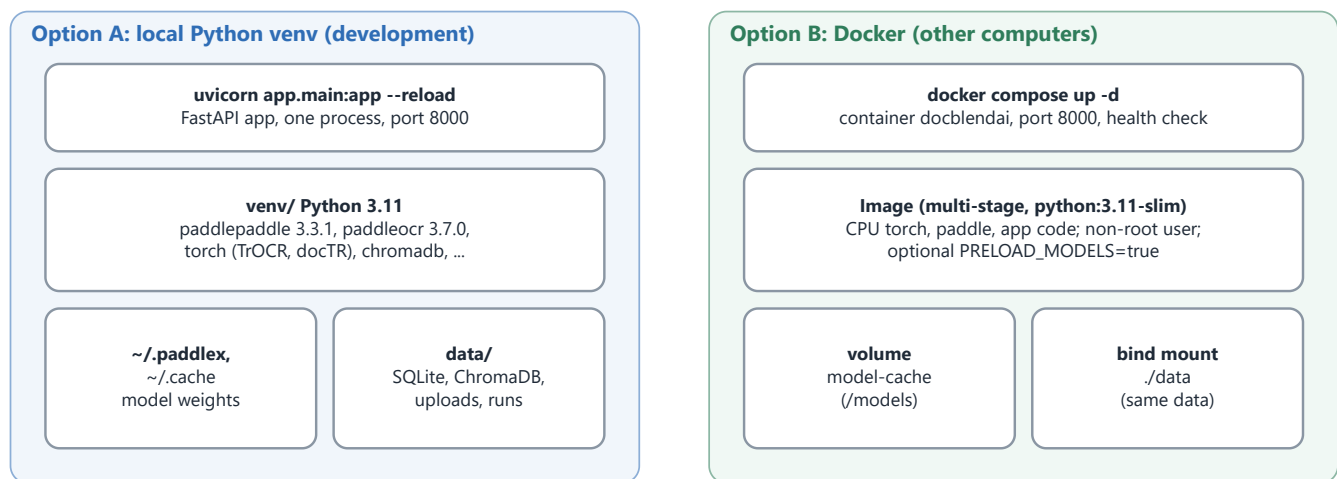


Figure 11. Two ways to run DocBlendAI.

Running

```
venv/Scripts/python -m pip install -r requirements.txt
venv/Scripts/python -m uvicorn app.main:app --reload          # http://localhost:8000

docker compose up -d                                         # same app in a container
```

The Gemini key is read from `.env` (`GEMINI_API_KEY`); model names and thresholds can be changed there too.

Testing

- About 440 automated tests (pytest) cover every module, router and the full upload → ask flow.
- Tests never call Gemini and never load real OCR/HTR models: fakes replace them (`tests/conftest.py`), so tests run fast and offline.
- A GitHub Actions workflow builds the Docker image.

Team workflow

`main` → feature branch → development → commit → push → pull request → review → merge. Each member owns modules; nobody commits directly to `main`. The build followed the synopsis order: typed path → embeddings → first end-to-end QA → OCR/HTR + confidence → content type + combined score + reliability → evaluation and UI.

17. Limitations and future work

Limitations

- OCR on CPU is slow for long scanned documents (about 1–1.5 minutes per page with PP-StructureV3).
- Answers need the Gemini API (internet and quota); free-tier quotas are small.
- Handwriting recognition works line by line and is weaker on very messy or mixed handwriting/print pages.
- Calibration is only as good as the labelled data used to fit it; more real labelled pages would improve it.
- Latin script only; no multilingual support.

Future work

- GPU support and page-parallel processing.
- A local open-source LLM option for offline use.
- Fine-tuning TrOCR on Indian student handwriting.
- A larger real evaluation set (scanned notes and handwritten answer sheets from the department).
- User accounts and per-user document libraries.

18. Glossary

OCR	Optical Character Recognition: reading printed text from images.
HTR	Handwritten Text Recognition.
RAG	Retrieval-Augmented Generation: retrieve relevant passages, then let an LLM answer from them.
Embedding	A vector of numbers representing the meaning of a text; similar meanings give similar vectors.
Cosine similarity	How closely two vectors point the same way (1 = identical direction).
Calibration	Adjusting confidence scores so they match the real probability of being correct.
ECE	Expected Calibration Error: the average gap between confidence and accuracy.
CER / WER	Character / Word Error Rate: edit distance divided by reference length.
Chunk	A piece of about 1000 characters of a document, the unit that is retrieved.
Transformer	Neural network built on self-attention; used by TrOCR and Gemini.
CTC	Connectionist Temporal Classification: decoding used by line recognisers.